Valenbisi IoT

Bicis en tiempo real: de la API a la web

Juan Andrés Molina Ascanio · Implantación de Aplicaciones Web · Práctica 3

¿Qué hace la aplicación?

1

Recoge

Consulta la API de Valenbisi y obtiene las 100 estaciones con sus bicis y huecos libres.

2

Transporta

Publica los datos por MQTT y los guarda en InfluxDB, una base de datos de series temporales.

3

Centraliza

Copia cada 15 segundos el histórico a PostgreSQL, en tablas relacionales.

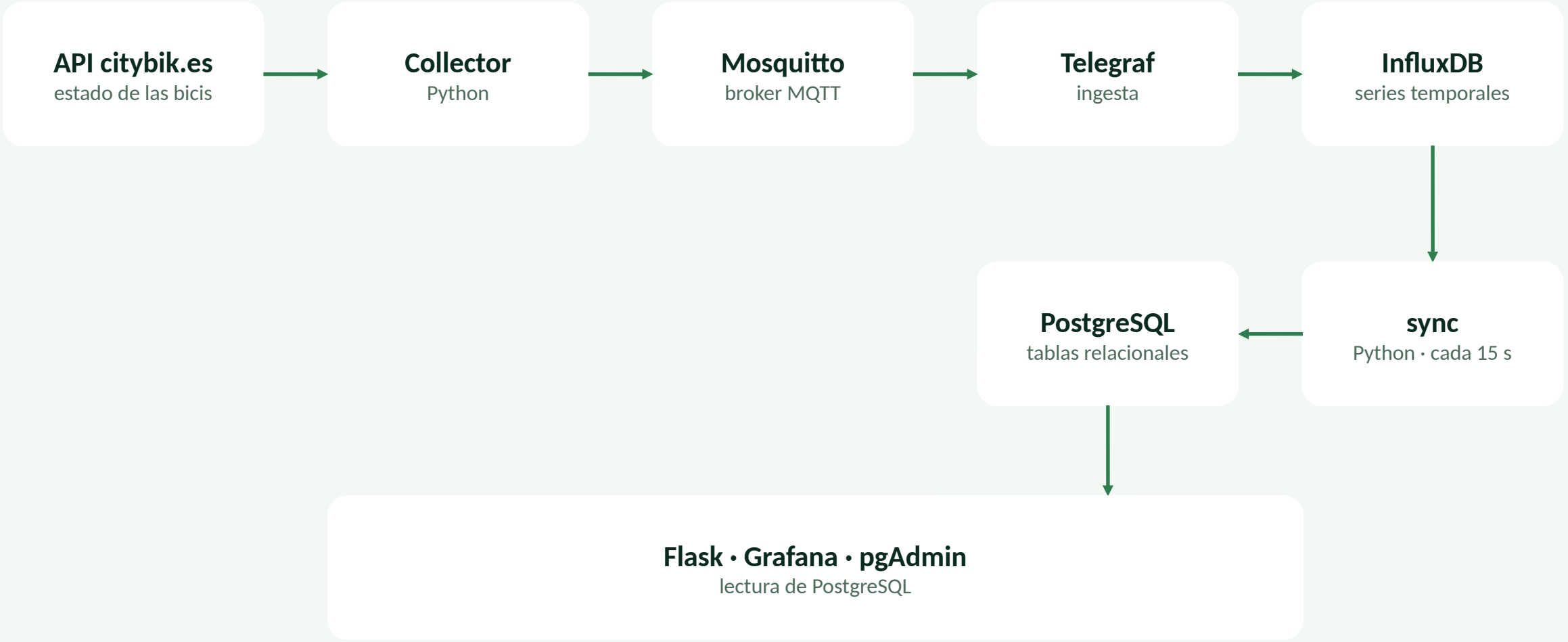
4

Muestra

Flask y Grafana presentan los datos con indicadores, gráficas y un mapa.

Todo se levanta con un solo comando: docker compose up -d --build

Flujo de datos



Tecnologías utilizadas

Python

requests, paho-mqtt, psycopg2 y Flask

Docker Compose

Levanta los 9 servicios con un solo comando

Mosquitto (MQTT)

Broker que transporta los mensajes del colector

Telegraf

Recoge los mensajes MQTT y los escribe en InfluxDB

InfluxDB 2 (Flux)

Base de datos de series temporales de las bicis

PostgreSQL 16

Estaciones, mediciones y registro de sincronizaciones

pgAdmin

Interfaz web para explorar y consultar PostgreSQL

Grafana

Dashboards con datasources de InfluxDB y PostgreSQL

Flask + Chart.js

Web con API JSON y gráficas en el navegador

El colector: de la API a MQTT

- Pide las 100 primeras estaciones a la API de citybik.es.
- Si la API limita o falla, espera 60 s y vuelve a intentarlo.
- Publica cada estación en el canal MQTT valenbisi/stations.

collector/collector.py (fragmento)

```
def obtener_estaciones():
    while True:
        try:
            r = requests.get(API_URL, timeout=10, headers=CABECERA)
            return r.json()["network"]["stations"][:100]
        except (requests.RequestException, ValueError, KeyError) as e:
            print(f"Error consultando la API: {e}", flush=True)
            time.sleep(ESPERA_ERROR)

    for e in estaciones:
        cliente.publish(MQTT_TOPIC, json.dumps(mensaje))
```

sync: de InfluxDB a PostgreSQL

- Lee InfluxDB con Flux en modo lectura: no modifica tus datos.
- Inserta en PostgreSQL y descarta los duplicados con ON CONFLICT.

Consulta Flux a InfluxDB

```
from(bucket: "valenbisi")
  |> range(start: desde)
  |> filter(fn: (r) => r._measurement == "mqtt_consumer")
  |> pivot(rowKey: ["_time", "station"], columnKey: ["_field"], valueColumn: "_value")
```

Inserción en PostgreSQL (sync.py)

```
execute_values(cur, """
    INSERT INTO mediciones (estacion_id, momento, bicis_libres, huecos_libres)
    VALUES %s ON CONFLICT (estacion_id, momento) DO NOTHING
""", datos)
```

Base de datos: tres tablas

estaciones

id (PK)
nombre (UNIQUE)
latitud
longitud

Catálogo: una fila por estación.

mediciones

id (PK)
estacion_id (FK)
momento
bicis_libres
huecos_libres
UNIQUE (estacion_id, momento)

Una fila por estación y minuto. No se repite.

sincronizaciones

id (PK)
ejecutado_en
filas_leidas
filas_insertadas

Registro de cada copia desde InfluxDB.

La restricción UNIQUE impide datos duplicados aunque sync se reinicie.

Flask: consulta y rutas JSON

- La consulta ACTUALES toma la última medición de cada estación.
- Cada ruta devuelve JSON que consume la página.

web/app.py · consulta base

```
SELECT DISTINCT ON (estacion_id) estacion_id, bicis_libres, huecos_libres
FROM mediciones
WHERE momento > now() - interval '15 minutes'
ORDER BY estacion_id, momento DESC
```

web/app.py · ruta del top 10

```
@app.route("/api/top-bicis")
def api_top_bicis():
    filas = consultar(f"""
        SELECT e.nombre AS estacion, u.bicis_libres AS bicis
        FROM estaciones e JOIN ({ACTUALES}) u ON u.estacion_id = e.id
        ORDER BY u.bicis_libres DESC LIMIT 10
    """)
```

El navegador dibuja las gráficas

- Pide los datos a las rutas JSON con fetch.
- Chart.js dibuja cada gráfica en el navegador.
- La página se actualiza sola cada 60 segundos.

web/templates/index.html (fragmento)

```
const [kpis, top] = await Promise.all([
  obtener('/api/kpis'), obtener('/api/top-bicis')]);
dibujar('g-top', {
  type: 'bar',
  data: { labels: top.map(r => r.estacion),
          datasets: [{ data: top.map(r => r.bicis) }] },
  options: { indexAxis: 'y' }
});
setInterval(cargar, 60000);   // se actualiza cada minuto
```

Grafana: datasources y paneles

- Los datasources de InfluxDB y PostgreSQL se configuran solos al arrancar (provisioning).
- Cada panel es una consulta sobre esos datos.
- Ejemplo: top 10 de estaciones con más bicis, en gráfico de barras.

Consulta del panel en PostgreSQL

```
SELECT e.nombre AS estacion, u.bicis_libres AS bicis
FROM estaciones e
JOIN (SELECT DISTINCT ON (estacion_id) estacion_id, bicis_libres
      FROM mediciones
      WHERE momento > now() - interval '15 minutes'
      ORDER BY estacion_id, momento DESC) u ON u.estacion_id = e.id
ORDER BY bicis DESC LIMIT 10;
```

Docker Compose: todo en un comando

- 9 servicios conectados en una red interna de Docker.
- Los nombres de servicio (postgres, influxdb) son los nombres de host.
- El volumen pgdata guarda PostgreSQL aunque se reinicie el contenedor.
- docker compose up -d --build levanta todo.

```
services:
  sync:
    build: ./sync
    depends_on: [influxdb, postgres]
    environment:
      INFLUX_URL: http://influxdb:8086
      PG_HOST: postgres
  web:
    build: ./web
    ports: ["5000:5000"]
```

Qué procesa el servidor y qué el cliente

Servidor

- Collector: obtiene y transforma los datos de la API.
- sync: agrega y copia los datos a PostgreSQL, sin duplicados.
- PostgreSQL: calcula la última medición de cada estación.
- Flask: genera las respuestas JSON y sirve la página.

Cliente

- El navegador ejecuta el script que pide los datos.
- Chart.js dibuja las gráficas y las actualiza cada minuto.
- Grafana renderiza sus paneles a partir de consultas SQL.
- pgAdmin permite explorar las tablas desde el navegador.

Problemas encontrados y solución

Problema	Solución
La API limita las peticiones si se piden cada 3 s	Pausa de 60 s y reintentos automáticos
Duplicados al guardar un lote de estaciones	Un diccionario por estación antes de insertar
Grafana no puede escribir en su carpeta	sudo chown -R 472:472 ./grafana
pgAdmin rechazaba el dominio .local	Usar admin@valenbisi.org
Contenedores con el mismo nombre	docker rm -f del contenedor antiguo

Conclusiones

- Datos en tiempo real de Valenbisi, guardados y consultables.
- Flujo completo: API → MQTT → InfluxDB → PostgreSQL → Flask y Grafana.
- Cubre el RA1: software, tecnologías y procesamiento en cliente y servidor.

Código y presentación

https://github.com/TU_USUARIO/Proyecto_IOT_VLC_Bicis